



JAVA PROGRAMMING

(340)

NATIONAL 2023

PRODUCTION:

Crypto Project Transfer _____ (460 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

Note to Graders:

- Output will be static for the data in each of the fields per the provided text files. All formatting should be present in the printing to console. The .csv file produced by the students will most likely open in a spreadsheet; however, you can also open the .csv file in a text document.
- The timestamp will vary amongst contestants' location and time setting on their computer. Please take this into consideration, especially if the code demonstrates they were able to extract the time from the computer or another resource. Hard coding the time is not acceptable.
- NOTE: we will not be checking for errors with bad sample files due to the complexity and length required.
- Ignore line spacing differences or grammar errors in the Field tags such Name, Quantity, etc. Some IDE's consoles will automatically wrap the text, please ignore these factors.
- The solution provided is just one of many possible ways to accomplish the challenge presented to the students. However, in the end they will need to read the text files, store their data, organize it, output the combined data into a central .csv file with matching print to console, and ensure inheritance and interfaces have been properly used.
- *When reading code: you might consider using the "National Solutiond.rtf" file (enclosed) due to the ease of its readability compared to the embedded solution at the end of this document which is heavily effected by the template margins.*

Test Case #1: program executes flawlessly and displays the data as displayed below.

NOTE: the order of the records will be in groups of four (1-4 Coins, 5-8 Tokens, 9-12 NFT).

```
*****
*****
Wed Dec 29 12:06:18 CST 2021
*****
```

```
Name: Bitcoin, Sym: BTC, Qty: 5.12E-4, CP: 48912.0, AP: 56234.0, BP: 59343.0
Name: Ethereum, Sym: ETH, Qty: 2.345, CP: 4323.0, AP: 4465.0, BP: 3738.0
Name: Cardano, Sym: ADA, Qty: 3245.0, CP: 1.34, AP: 2.21, BP: 2.09
Name: Dogecoin, Sym: DOGE, Qty: 33922.0, CP: 0.1595, AP: 0.2343, BP: 0.2386
Name: Polygon, Sym: MATIC, Qty: 56.34, CP: 2.34, AP: 1.84, BP: 0.34
Name: Sandbox, Sym: SAND, Qty: 1222.0, CP: 5.87, AP: 2.77, BP: 2.72
Name: Shiba-Inu, Sym: SHIB, Qty: 1345987.0, CP: 3.395E-5, AP: 5.645E-5, BP: 2.35E-5
Name: Uniswap, Sym: UNI, Qty: 234.2, CP: 14.75, AP: 11.23, BP: 5.82
Name: Ethicorns, Sym: BNB, Qty: 6.334, CP: 5.34, AP: 4.34, BP: 5.67
Name: ChainSports, Sym: ETH, Qty: 0.2, CP: 1.2, AP: 2.2, BP: 0.3
Name: MetaLands, Sym: MATIC, Qty: 1.645, CP: 0.345, AP: 3.2123, BP: 1.4
Name: League of Heroes, Sym: MATIC, Qty: 0.45, CP: 4.2, AP: 2.12, BP: 2.98
```

Solution and Project		
The project is present on the flash drive	_____	10 points
The projects main class is named CryptoProjectTransfer	_____	10 points
The parent class of Coins, Tokens, NFT classes is InvestmentProducts (10 pts each)	_____	30 points
InvestmentProducts class properly applies the Assett_Interface	_____	20 points
MyWrapper class properly applies the Wrapper_Interface	_____	20 points
RecordReader class properly applies the FileReader_Interface	_____	20 points
Program Execution		
The program runs from the USB flash drive	_____	10 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
The final printout to console and the text file match (excluding font size and types); this includes abbreviations, colons, commas, and data (1 error = no points)	_____	20 points
The order of the fields in the records printed matches the proper order: (1)Name; (2)Symbol; (3)Quantity; (4)Current Price; (5) Avg. Price; (6) Buy Price	_____	20 points
The order of the records printed matches the proper order: (1)Coins; (2)Tokens; (3)NFT	_____	20 points
The final outputted file is called “Portfolio.csv” and is stored in the same folder as the driver class.	_____	10 points
Source Code Review		
The source code is properly commented	_____	10 points
A comment containing the contestant number is present at the top of the program	_____	10 points
InvestmentProducts class uses the Assett_Interface required methods; you will not be penalized for bypassing them and creating alternative solutions	_____	30 points
Coins, Tokens, and NFT class constructors call super () with parameters to be created with InvestmentProducts class (10 points each)	_____	30 points
InvestmentProducts class toString() method is used to print all detail for its objects.	_____	10 points
MyWrapper class uses the Wrapper_Interface required methods; you will not be penalized for bypassing them and creating alternative solutions	_____	30 points
MyWrapper class contains a data structure that can host all three types of investment products	_____	20 points

The fileWriter() method in MyWrapper class is used to write the records to the final “Portfolio.csv” file.	_____	30 points
The exportDate(PrintWriter out) method in MyWrapper class is used to create the date time stamp for the file and the print to console.	_____	20 points
The openWriter(String name) method in MyWrapper class is used create the Portfolio.csv file	_____	30 points
RecordReader class uses the FileReader_Interface required methods; you will not be penalized for bypassing them and creating alternative solutions	_____	30 points
readBlock(BufferedReader in) method in the RecordReader class reads and returns the proper investment object type.	_____	20 points
Total Points		/460 points

Possible Solution

```

1
2
3 import java.util.*;
4 import java.text.NumberFormat;
5 import java.io.*;
6
7
8 public class CryptoProjectTransfer
9 {
10
11     public static void main (String args [])
12     {
13
14         //All three instances of RecordReader below reference the same wrapper
object
15         MyWrapper wrp = new MyWrapper();
16
17         //This is one of the easiest tactics and most probable, but not the only
way
18         RecordReader rC = new RecordReader(wrp, "Coins.txt");
19         RecordReader rT = new RecordReader(wrp, "Tokens.txt");
20         RecordReader rN = new RecordReader(wrp, "NFT.txt" );
21
22         //This is one of the easiest tactics and most probable, but not the only
way
23         //It allows all of the data to get read off of the three text files and
24         //consolodated into the wrapper arraylist.
25         rC.inputFileReader();
26         rT.inputFileReader();
27         rN.inputFileReader();
28
29         wrp.fileWriter();
30     }
31 }
32
33 ///////Assett Objects Information
34 //!!STUDENT WILL MAKE THIS <===
35 class InvestmentProducts implements Assett_Interface
36 {
37
38     private String type;
39     private String name;
40     private String symbol;
41     private double quantity;
42     private double currentPrice;
43     private double avgPrice;
44     private double buyPrice;
45
46     //1st Constructor that takes in the type of investment symbol. We don't use
this in the file output
47     public InvestmentProducts(String type, String name, String symbol, double
quantity, double currentPrice,
48                             double avgPrice, double buyPrice)
49     {

```

```
50     this.type = type;
51     this.name=name;
52     this.symbol=symbol;
53     this.quantity=quantity;
54     this.currentPrice=currentPrice;
55     this.avgPrice=avgPrice;
56     this.buyPrice=buyPrice;
57 }
58
59 //2nd Constructor that ignores the type of investment symbol. We use this in
the program
60     public InvestmentProducts(String name, String symbol, double quantity, double
currentPrice,
61                               double avgPrice, double buyPrice)
62     {
63         this.type = "";
64         this.name=name;
65         this.symbol=symbol;
66         this.quantity=quantity;
67         this.currentPrice=currentPrice;
68         this.avgPrice=avgPrice;
69         this.buyPrice=buyPrice;
70     }
71
72 //Required Get and Set functions per the interface implementation requirements
73 //Set functions will not be used in the file output.
74     public String getType()
75     {
76         return type;
77     }
78     public void setType(String t)
79     {
80         type = t;
81     }
82     public String getName()
83     {
84         return name;
85     }
86     public void setName(String n)
87     {
88         name = n;
89     }
90     public String getSymbol()
91     {
92         return symbol;
93     }
94     public void setSymbol(String s)
95     {
96         symbol = s;
97     }
98     public double getQuantity()
99     {
100         return quantity;
101     }
102     public void setQuantity(double q)
103     {
```

```
104     quantity = q;
105 }
106 public double getCurrentPrice()
107 {
108     return currentPrice;
109 }
110 public void setCurrentPrice(double cp)
111 {
112     currentPrice = cp;
113 }
114 public double getAvgPrice()
115 {
116     return avgPrice;
117 }
118 public void setAvgPrice(double ap)
119 {
120     avgPrice = ap;
121 }
122 public double getBuyPrice()
123 {
124     return buyPrice;
125 }
126 public void setBuyPrice(double bp)
127 {
128     buyPrice = bp;
129 }
130
131 public String toString()
132 {
133
134     return "Name: " +getName()+ ", Sym: " + getSymbol()+ ", Qty: " +
getQuantity()+ ", CP: " +
135     getCurrentPrice()+ ", AP: " + getAvgPrice()+ ", BP: " + getBuyPrice()+ "
\n";
136 }
137
138
139 }
140
141 ///!!STUDENT WILL MAKE THIS <===
142 class NFT extends InvestmentProducts
143 {
144     //These are basic and call super to construct and overrides toString
inclusively
145     public NFT(String n, String s, double q, double cp, double ap, double bp)
146     {
147         super(n,s,q,cp,ap,bp);
148     }
149
150 }
151
152 ///!!STUDENT WILL MAKE THIS <===
153 class Coins extends InvestmentProducts
154 {
155     //These are basic and call super to construct and overrides toString
inclusively
```

```

156     public Coins(String n, String s, double q, double cp, double ap, double bp)
157     {
158         super(n,s,q,cp,ap,bp);
159     }
160
161
162 }
163
164 ///!!STUDENT WILL MAKE THIS <==
165 class Tokens extends InvestmentProducts
166 {
167
168     //These are basic and call super to construct and overrides toString
inclusively
169     public Tokens(String n, String s, double q, double cp, double ap, double bp)
170     {
171         super(n,s,q,cp,ap,bp);
172     }
173
174 }
175
176 //Wrapper Class////////////////////////////////////
177 //Uses a generic ArrayList for the three types of investment objects//
178 ///!!STUDENT WILL MAKE THIS <=====
179 class MyWrapper implements Wrapper_Interface
180 {
181     //Generic ArrayList: for the given circumstances of this program
182     //it can store any object file type; this is one of many options
183     ArrayList arraylist = new ArrayList();
184     String temp = ""; //This instance variable will host all records for printing
185
186     public MyWrapper()
187     {
188
189     }
190     public void addToList(Object a)
191     {
192         arraylist.add(a);
193     }
194     //Test prints the objects to console
195     //Not Required method...just one of many ways
196     public void wrapperPrinter(String s)
197     {
198         System.out.print(s);
199     }
200
201
202     //Builds single String object with delimiters and headers
203     //from generic structure. Called from fileTextReturn()
204     //Not Required...just one of many ways
205     public void fileFeed()
206     {
207         for(Object o : arraylist)
208         {
209             String temp2 = o.toString();
210             temp += temp2;

```

```

211     }
212
213 }
214 //Calls the functions that builds the String object and returns it
215 //Not Required
216 public String fileTextReturn()
217 {
218     fileFeed();
219     return "\n"+ temp; //instance String from above
220 }
221
222 ///////////////////////////////////////////////////////////////////
223 //Writes to the PrintWriter file object by using the fileTextReturn and
fileFeed methods.
224 //fileTextReturn and fileFeed methods are not required but this method MUST
write to the file object
225 //Interface method
226 ///!!STUDENT WILL MAKE THIS <=====
227 public void fileWriter()
228 {
229     PrintWriter out = openWriter("Portfolio.csv");
230     exportDate(out); //This creates a consistent format for console and file
231     String wraptemp = fileTextReturn(); //Not required method there's other
solutions besides this
232     wrapperPrinter(wraptemp); //Not required method there's other solutions
besides this
233     out.println(wraptemp); //This will have to happen to get the data to the
file
234     out.close(); //It's okay if they forget to close; not part of the rubric
235 }
236
237 ///////////////////////////////////////////////////////////////////
238 //Prints the current date to console and to the file
239 //Interface method
240 ///!!STUDENT WILL MAKE THIS <=====
241 public void exportDate(PrintWriter out)
242 {
243     java.util.Date date = new java.util.Date();
244     //Console print
245     System.out.println("\n*****");
246     System.out.println("*****");
247     System.out.println(date+"\n*****");
248     //.csv file print
249     out.println("\n*****");
250     out.println("*****");
251     out.println(date+"\n*****");}
252
253 ///////////////////////////////////////////////////////////////////
254 //Creates the PrintWrite object file
255 //Interface method
256 ///!!STUDENT WILL MAKE THIS <=====
257 public PrintWriter openWriter(String name)
258 {
259     try
260     {
261         File file = new File(name);

```

```

262         PrintWriter out =
263             new PrintWriter(
264                 new BufferedWriter(
265                     new FileWriter(file) ), true );
266         return out;
267     }
268     catch (IOException e)
269     {
270         System.out.println("I/O Error");
271         System.exit(0);
272     }
273     return null;
274 }
275 }
276 }
277 }
278
279 //File Input////////////////////////////////////
280 //!!STUDENT WILL MAKE THIS <=====
281 class RecordReader implements FileReader_Interface
282 {
283     //These are not required
284     private BufferedReader in;
285     MyWrapper wrp;
286     Object obj;
287
288     //The constructor setup below is not required
289     //NOTE: this MyWrapper object is still through reference via Main()
290     public RecordReader(MyWrapper w, String r)
291     {
292         wrp = w;
293         in = getReader(r);
294         obj = null;
295     }
296
297
298     //!!STUDENT WILL MAKE THIS <=====
299     //This method is called from the object in the Main class
300     public void inputFileReader()
301     {
302         obj = readBlock(in); //old one was readCustomer
303         wrp.addToList(obj);
304         while ( obj != null)
305         {
306             obj = readBlock(in);
307             if( obj != null)
308                 wrp.addToList(obj);
309         }
310     }
311 }
312
313 //////////////////////////////////////
314 //PARTIAL STUDENT AND GIVEN
315
316 //GIVEN
317 public Object readBlock(BufferedReader in)

```

```
318     {
319
320         String type = "";
321         String name = "";
322         String symbol = "";
323         double quantity;
324         double currentPrice;
325         double avgCost;
326         double buyPrice;
327
328
329         String line = "";
330         String[] data;
331
332         //catches errors in the read
333         try
334         {
335             line = in.readLine();
336         }
337         catch (IOException e)
338         {
339             System.out.println("I/O Error");
340             System.exit(0);
341         }
342         //terminates the file read once it gets to the end
343         if (line == null)
344             return null;
345         else
346         {
347             //each field goes in escalating arithmetic sequence 0 to 6
348             data = line.split(";");
349             type = data[0];
350             name = data[1];
351             symbol = data[2];
352             quantity = Double.parseDouble(data[3]);
353             currentPrice = Double.parseDouble(data[4]);
354             avgCost = Double.parseDouble(data[5]);
355             buyPrice = Double.parseDouble(data[6]);
356
357             //NOTE: removed this from the following condition checks "type.trim(),"
358             //!!!STUDENT WILL MAKE THIS <=====
359
360             type = type.trim();
361             //There's other ways to accomplish this
362             //Code will have to be inspected to see if they create specific objects
363             if(type.equals("C"))
364             {
365                 return new Coins(name.trim(), symbol.trim(), quantity, currentPrice,
366                 avgCost, buyPrice);
367             }
368             else if(type.equals("T"))
369             {
370                 return new Tokens(name.trim(), symbol.trim(), quantity, currentPrice,
371                 avgCost, buyPrice);
372             }
373             else
```

```

372         {
373             return new NFT(name.trim(), symbol.trim(), quantity, currentPrice,
374                             avgCost, buyPrice);
375         }
376     }
377 }
378 }
379 ///////////////////////////////////////////////////
380 //GOOD LEAVE ALONE
381 //GIVEN
382 public BufferedReader getReader(String name)
383 {
384     BufferedReader in = null;
385     try
386     {
387         File file = new File(name);
388         in = new BufferedReader(
389             new FileReader(file) );
390     }
391     catch (FileNotFoundException e)
392     {
393         System.out.println(
394             "The file doesn't exist.");
395         System.exit(0);
396     }
397     return in;
398 }
399
400 }
401
402 ///////Interfaces/////
403
404 //GIVEN
405 interface Assett_Interface
406 {
407     public String getType();
408     public void setType(String t);
409     public String getName();
410     public void setName(String n);
411     public String getSymbol();
412     public void setSymbol(String s);
413     public double getQuantity();
414     public void setQuantity(double q);
415     public double getCurrentPrice();
416     public void setCurrentPrice(double cp);
417     public double getAvgPrice();
418     public void setAvgPrice(double ap);
419     public double getBuyPrice();
420     public void setBuyPrice(double bp);
421     public String toString();
422
423
424 }
425
426 interface FileReader_Interface

```

```
427 {  
428     public void inputFileReader(); //Pass this the text file for the  
429     public Object readBlock(BufferedReader in);  
430     public BufferedReader getReader(String name);  
431 }  
432 interface Wrapper_Interface  
433 {  
434     public void exportDate(PrintWriter out); //PrintWriter out  
435     public PrintWriter openWriter(String w); //String name  
436     public void fileWriter();  
437 }
```